

Frontend Team Document

Context Compression Engine — Frontend Build Guide

Your role: Build everything the user sees and interacts with — input screen, loading states, and the results dashboard that makes the compression + accuracy proof visually convincing to judges.

1. Your Responsibilities

- Build the input page (paste/upload context)
- Build the compression trigger flow + loading states
- Build the metrics dashboard (compression ratio, cost saved, latency, accuracy)
- Build the before/after diff view
- Build the accuracy validation side-by-side view
- Build a simple history panel
- Connect all of the above to the backend API routes

2. Tech Stack You'll Use

- Next.js 14 (App Router)
- React (function components, hooks)
- Tailwind CSS for styling
- A diff-rendering approach (simple: highlight removed lines with strikethrough/red, kept lines normal — can hand-roll this by comparing line arrays, no need for a heavy library)

3. Step-by-Step Build Plan

Step 1: Project Setup (30 min)

1. Confirm Next.js project is initialized (coordinate with Backend team — likely shared repo)

2. Set up Tailwind config

3. Create folder structure:

```
/app/page.tsx                -> main app page
/components/InputPanel.tsx
/components/CompressButton.tsx
/components/MetricsDashboard.tsx
/components/DiffView.tsx
/components/AccuracyComparison.tsx
/components/HistoryPanel.tsx
/components/LoadingState.tsx
/lib/api.ts                   -> fetch wrapper functions
```

Step 2: Design Direction (30–45 min)

Before writing components, decide the visual language (this matters a lot for a hackathon demo — judges remember a clean UI):

- Follow monochrome + accent-color pattern (consistent with your other InnovaHacks project): dark background, white/gray text, and reserve a color accent (e.g., green/amber/red) specifically for the compression ratio and accuracy score — so the “proof” numbers visually pop
- Keep layout: input at top, “Compress” button, then results dashboard appears below with a smooth transition (no page reload — all client-side state)

Step 3: Input Panel (1 hr)

Build `components/InputPanel.tsx`:

1. A large `<textarea>` for pasting raw text (character counter shown live)
2. A file upload button (`<input type="file" accept=".txt,.log,.md,.js,.py,.ts">`) — on file select, read file as text via `FileReader` and populate the textarea
3. An optional text input for “Test question (optional — we’ll generate one if left blank)”
4. Basic client-side validation: disable submit if textarea is empty, show a warning if text exceeds ~50,000 characters

Step 4: API Wrapper (30 min)

Build `lib/api.ts`:

1. `compressText(originalText: string) → POST /api/compress`, returns parsed JSON,

throws on non-200

2. `validateAccuracy(originalText, compressedText, question)` → POST `/api/validate`, returns parsed JSON
3. `fetchHistory()` → GET `/api/history`, returns parsed JSON
4. Centralize error handling here — return a consistent `{ data, error }` shape so components don't need try/catch everywhere

Important: agree on the exact request/response field names with the Backend team before building this file — use the contracts in the SRS doc as your source of truth.

Step 5: Main Page Orchestration (1 hr)

Build `app/page.tsx` as the state manager:

1. **State:** `originalText`, `question`, `compressResult`, `validateResult`, `loadingStage` (`'idle' | 'compressing' | 'validating' | 'done' | 'error'`), `history`
2. On “Compress” click:
 - Set `loadingStage = 'compressing'`
 - Call `compressText()` → store result → show Stage 1/Stage 2 metrics as soon as available
 - Set `loadingStage = 'validating'`
 - Call `validateAccuracy()` → store result
 - Set `loadingStage = 'done'`
3. On mount, call `fetchHistory()` to populate the history panel
4. Pass state down to child components as props

Step 6: Loading States (45 min)

Build `components/LoadingState.tsx`:

1. Show distinct messages per stage so the demo feels transparent and alive: “Stripping structural redundancy...” → “Scoring chunks for relevance...” → “Compressing low-priority sections...” → “Validating accuracy against original...”
2. Use a simple animated progress indicator (not a fake progress bar tied to time — tie it to `loadingStage` transitions)

Step 7: Metrics Dashboard (1.5 hrs)

Build `components/MetricsDashboard.tsx`. This is the most important visual component — it's your "proof" screen.

1. Four stat cards: **Original Tokens**, **Compressed Tokens**, **Compression Ratio %**, **Estimated Cost Saved**
 - $\text{Cost saved} = (\text{originalTokens} - \text{finalTokens}) * \text{pricePerToken}$ — get current Claude pricing from Backend/shared constant, display as `$X.XX per request` and optionally extrapolate to "at 10,000 requests/day"
2. A large, prominent **Compression Ratio** number with a colored badge (green if $\geq 70\%$, matching the target from the PRD)
3. A **Latency Comparison** bar chart or simple two-bar visual: full-context response time vs compressed-context response time
4. Make numbers animate/count up on render for demo polish (simple `useEffect` + `interval`, or a lightweight counter hook)

Step 8: Diff View (1.5 hrs)

Build `components/DiffView.tsx`:

1. Take `originalText` and `compressedText` as props (or `stage1Text` for an intermediate view — offer a toggle: "Original → Stage 1" / "Stage 1 → Final")
2. Split both into lines, do a simple line-level comparison:
 - Lines present in original but not in compressed → render with strikethrough + red-tinted background
 - Lines present in both → render normal
 - (Optional, if time allows) Lines that were summarized → render in amber to indicate "modified"
3. Keep this scrollable within a fixed-height container — don't let it blow up page length uncontrollably
4. This doesn't need to be a perfect diff algorithm — a simple set-difference on lines is enough for a convincing demo

Step 9: Accuracy Comparison View (1 hr)

Build `components/AccuracyComparison.tsx`:

1. Two side-by-side (or stacked on mobile) cards: "Answer using FULL context" and "Answer using COMPRESSED context"
2. Show the test question above both
3. Show the similarity/retention score prominently between them (e.g., "96% Match" badge, green/amber/red based on threshold)
4. This is the component judges will screenshot — make it clean and legible

Step 10: History Panel (45 min)

Build `components/HistoryPanel.tsx`:

1. Simple list/table: timestamp, compression ratio, accuracy score for each past run
2. Click a row to reload that run's data into the dashboard (optional stretch goal — skip if time is short)

Step 11: Polish Pass (last 1–2 hrs before demo)

1. Responsive check — at minimum works well on a laptop screen for the live demo
2. Add empty states (before first compression, show a placeholder/explainer instead of blank dashboard)
3. Add error states (if API call fails, show a clear message, not a blank screen)
4. Double check all numbers are formatted (commas in token counts, % with 1 decimal, \$ with 2 decimals)

4. What "Done" Looks Like

- User can paste/upload text, hit Compress, and see a full dashboard within ~20 seconds
- Dashboard clearly shows compression ratio, cost saved, and accuracy retention
- Diff view and accuracy comparison are clean enough to screenshot for the pitch deck
- No blank/broken states during a live demo run-through

5. Coordination Notes

- Don't wait on Backend to finish the real pipeline — request a stub JSON response matching the agreed contract on Day 1, Hour 1, and build against that
- Confirm field names match exactly what Backend returns (case-sensitive) before wiring

up real calls

- Do a full end-to-end run-through with Backend at least 2 hours before the deadline to catch integration bugs